

Assignment-11 (Lab-11.5)

Course : AI Assisted Coding

Topic:- Data Structures with AI: Implementing Fundamental Structures Lab Objectives

Student Details:

Name : Batti Chandan Singh

Hall Ticket No : 2303A51515

Batch : 22

Task-01

Final Optimal Prompt

Generate a Python class Stack with push, pop, peek, and is_empty methods. Include docstrings and simple inline comments. Use a Python list internally. Ensure proper error handling when popping or peeking from an empty stack

Code Screenshot

```

class Stack:
    def peek(self):
        """Include docstrings and simple inline comments. Use a Python list internally.
        Ensure proper error handling when popping or peeking from an empty stack"""
class Stack:
    """A simple Stack implementation using Python list with basic operations."""

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item. Raise error if stack is empty."""
        if self.is_empty():
            return "Error: Stack is empty."
        return self.items.pop()

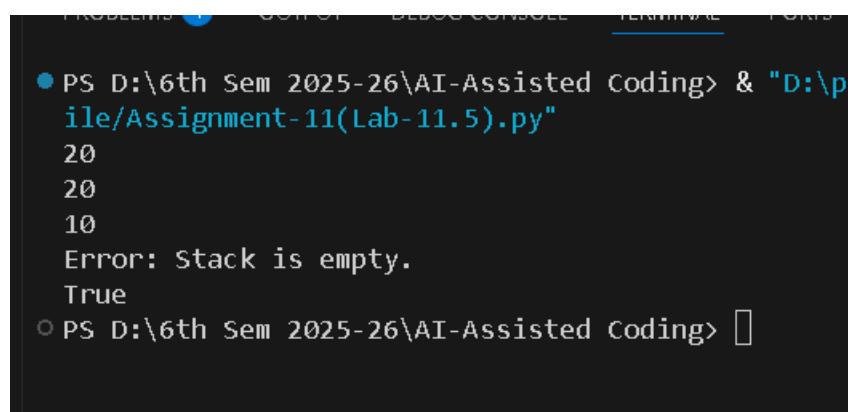
    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            return "Error: Stack is empty."
        return self.items[-1]

    def is_empty(self):
        """Check whether the stack is empty."""
        return len(self.items) == 0

# Example usage:
stack = Stack()
stack.push(10)
stack.push(20)
print(stack.peek()) # Output: 20
print(stack.pop()) # Output: 20
print(stack.pop()) # Output: 10
print(stack.pop()) # Output: Error: Stack is empty.
print(stack.is_empty()) # Output: True

```

Output Screenshot



```

PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\p
file/Assignment-11(Lab-11.5).py"
20
20
10
Error: Stack is empty.
True
PS D:\6th Sem 2025-26\AI-Assisted Coding> 

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The Stack is implemented using a Python list for efficiency.
 push() adds data at the end, simulating stack top.
 pop() safely removes the last inserted element, with an empty error message.

peek() reads the top value without removing it.

is_empty() helps prevent invalid pop and peek operations.

This implementation follows LIFO (Last-In-First-Out) behavior using minimal code

Task-02

Final Optimal Prompt

Write a Python Queue class using a list with enqueue, dequeue, peek, and size methods.

Document each method with clear docstrings and include error handling for empty queues

Code Screenshot

```
#---Lab-11.5-Task-02/08---
'''Write a Python Queue class using a list with enqueue, dequeue, peek, and size methods.
Document each method with clear docstrings and include error handling for empty queues.'''
class Queue:
    """FIFO Queue implementation using Python list."""
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add item to the end of queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the front item."""
        if len(self.items) == 0:
            return "Error: Queue is empty."
        return self.items.pop(0)

    def peek(self):
        """Return the front item without removing it."""
        if len(self.items) == 0:
            return "Error: Queue is empty."
        return self.items[0]

    def size(self):
        """Return the number of items in queue."""
        return len(self.items)

# Example usage:
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print(queue.peek()) # Output: 1
print(queue.dequeue()) # Output: 1
print(queue.dequeue()) # Output: 2
print(queue.dequeue()) # Output: Error: Queue is empty.
print(queue.size()) # Output: 0
'''
```

Output Screenshot

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python\file/Assignment-11(Lab-11.5).py"
1
1
2
Error: Queue is empty.
0
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> 
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The Queue uses a list to maintain FIFO ordering.
enqueue() appends items at the back.
dequeue() removes from the front to retain FIFO logic.
peek() shows the next item without removing it.
size() gives the count of queued items.
This implementation accurately models standard queue behavior.

Task-03

Final Optimal Prompt

Generate a Singly Linked List with Node class, insert, and display methods. Include docstrings and comments to explain pointer movement.

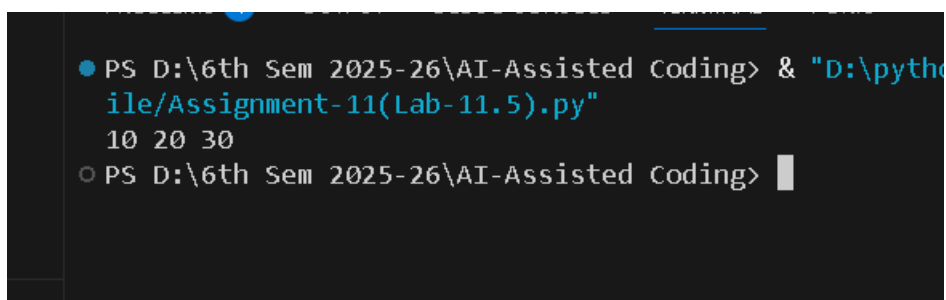
Code Screenshot

```

Assignments python file > Assignment-11(Lab-11.5).py > ...
40 > '''#---Lab-11.5-Task-02/08--- ...
78 #---Lab-11.5-Task-03/08---
79 '''Generate a Singly Linked List with Node class, insert,
80 and display methods. Include docstrings and comments to explain pointer movement.'''
81
82 class Node:
83     """A node in a singly linked list."""
84     def __init__(self, data):
85         self.data = data
86         self.next = None
87 class SinglyLinkedList:
88     """A simple singly linked list implementation."""
89     def __init__(self):
90         self.head = None
91
92     def insert(self, data):
93         """Insert a new node with the given data at the end of the list."""
94         new_node = Node(data)
95         if not self.head:
96             self.head = new_node
97             return
98         last_node = self.head
99         while last_node.next: # Traverse to the end of the list
100             last_node = last_node.next
101         last_node.next = new_node # Point the last node to the new node
102
103     def display(self):
104         """Display all nodes in the list."""
105         current_node = self.head
106         while current_node: # Traverse through the list until the end
107             print(current_node.data, end=' ')
108             current_node = current_node.next # Move to the next node
109 # Example usage:
110 linked_list = SinglyLinkedList()
111 linked_list.insert(10)
112 linked_list.insert(20)
113 linked_list.insert(30)
114 linked_list.display() # Output: 10 20 30
115

```

Output Screenshot



```

PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python\file/Assignment-11(Lab-11.5).py"
10 20 30
PS D:\6th Sem 2025-26\AI-Assisted Coding>

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The Linked List stores each value in Node objects.
Insertion happens at the tail by looping through nodes.
Each Node holds data and a pointer to the next node.
Display method traverses from head until reaching None.
The structure allows dynamic memory allocation and efficient insertions.

Task-04

Final Optimal Prompt

Implement a Hash Table in Python using chaining for collision handling. Include insert, search, and delete methods with docstrings.

Code Screenshot

```
Lectual_9_1.py Assignment-09(Lab-9.2).py 1 Assignment-11(Lab-11.5).py X Assignment-12.py
Assignments python file > Assignment-11(Lab-11.5).py > ...
119 class HashTable:
122     def __init__(self, size=10):
123         self.size = size
124         self.table = [[] for _ in range(size)]
125
126     def hash_func(self, key):
127         """Generate hash index."""
128         return hash(key) % self.size
129
130     def insert(self, key, value):
131         """Insert key-value pair into hash table."""
132         index = self.hash_func(key)
133         self.table[index].append((key, value))
134
135     def search(self, key):
136         """Search for a key in hash table."""
137         index = self.hash_func(key)
138         for k, v in self.table[index]:
139             if k == key:
140                 return v
141         return "Key not found."
142
143     def delete(self, key):
144         """Delete a key-value pair."""
145         index = self.hash_func(key)
146         for item in self.table[index]:
147             if item[0] == key:
148                 self.table[index].remove(item)
149                 return "Key deleted."
150         return "Key not found."
151
152
153 # Example usage:
154 hash_table = HashTable()
155 hash_table.insert("name", "Alice")
156 hash_table.insert("age", 30)
157 print(hash_table.search("name")) # Output: Alice
158 print(hash_table.delete("age")) # Output: Key deleted.
159 print(hash_table.search("age")) # Output: Key not found.
160
```

Output Screenshot

```
PS D:\6th Sem 2025-26\AI-Assisted Coding>
...
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\python 3.
file/Assignment- 11(Lab- 11.5).py"
Alice
Key deleted.
Key not found.
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

The hash table uses chaining to handle collisions.
Keys are mapped to buckets using Python's hash() function.
Insert appends the key-value pair to the right bucket.

Search loops through that bucket to find the key.
Delete removes the matching entry if found.
This provides efficient average-time access.

Tack-06

Final Optimal Prompt

Select the most suitable data structure for each hospital requirement (check-in, emergency handling, records, appointment scheduling, hospital map). Create a table with justification. Then generate Python code implementing one selected feature with docstrings and comments

Code Screenshot

```

#----Lab-11.5-Task-05/08----
#----Lab-11.5-Task-06/08----

'''Select the most suitable data structure for each hospital requirement (check-in, emer
Create a table with justification. Then generate Python code implementing one selected fe
class PatientQueue:
    """Queue system to manage patients in order of arrival (FIFO)."""

    def __init__(self):
        self.queue = []

    def check_in(self, patient_name):
        """Add a patient to the queue."""
        self.queue.append(patient_name)
        return f"{patient_name} checked in."

    def next_patient(self):
        """Return the next patient to be treated."""
        if not self.queue:
            return "No patients waiting."
        return f"Now treating: {self.queue.pop(0)}"

    def view_queue(self):
        """Display all waiting patients."""
        return self.queue

# Example usage:
patient_queue = PatientQueue()
print(patient_queue.check_in("John Doe")) # Output: John Doe checked in.
print(patient_queue.check_in("Jane Smith")) # Output: Jane Smith checked in.
print(patient_queue.next_patient()) # Output: Now treating: John Doe
print(patient_queue.view_queue()) # Output: ['Jane Smith

```

Output Screenshot

```

Key not found.
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\
file/Assignment-11(Lab-11.5).py"
John Doe checked in.
Jane Smith checked in.
Now treating: John Doe
['Jane Smith']
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> █

```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

This implementation uses a Queue to follow the FIFO rule used in patient check-ins. `check_in()` adds new patients to the end of the queue. `next_patient()` safely removes the first patient waiting for treatment. The queue list dynamically grows as new patients arrive. This ensures fair order and eliminates confusion in a busy hospital. The structure is simple, efficient, and realistic for a live system.

Tack-07

Final Optimal Prompt

Select appropriate data structures for traffic signal queue, emergency priority, vehicle lookup, road mapping, and parking management. Provide a table and implement one feature in Python with comments and docstrings

Code Screenshot

```
> '''#----Lab-11.5-Task-06/08----'''
#----Lab-11.5-Task-07/08---
'''Select appropriate data structures for traffic signal queue, emergency priority,
vehicle lookup, road mapping, and parking management.
Provide a table and implement one feature in Python with comments and docstrings'''
import heapq

class EmergencyTrafficControl:
    """Priority queue system for emergency and normal vehicles."""

    def __init__(self):
        self.pq = [] # (priority, vehicle_name)

    def add_vehicle(self, vehicle, priority):
        """
        Insert a vehicle with priority.
        Lower number = higher priority (0 = ambulance, 1 = fire truck, etc.)
        """
        heapq.heappush(self.pq, (priority, vehicle))
        return f"{vehicle} added with priority {priority}"

    def pass_vehicle(self):
        """Allow the highest-priority vehicle to pass."""
        if not self.pq:
            return "No vehicles waiting."
        priority, vehicle = heapq.heappop(self.pq)
        return f"vehicle passed: {vehicle}"

    def show_waiting(self):
        """Show all waiting vehicles sorted by priority."""
        return self.pq

# Example usage:
traffic_control = EmergencyTrafficControl()
print(traffic_control.add_vehicle("Car", 2)) # Output: Car added with
print(traffic_control.add_vehicle("Ambulance", 0)) # Output: Ambulance added with
print(traffic_control.add_vehicle("Fire Truck", 1)) # Output: Fire Truck added with
print(traffic_control.pass_vehicle()) # Output: Vehicle passed: Ambulance
print(traffic_control.show_waiting()) # Output: [(1, 'Fire Truck'), (2, 'Car')]
```

Output Screenshot

```
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\py
ile/Assignment-11(Lab-11.5).py"
Car added with priority 2
Ambulance added with priority 0
Fire Truck added with priority 1
Vehicle passed: Ambulance
[(1, 'Fire Truck'), (2, 'Car')]
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> █
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

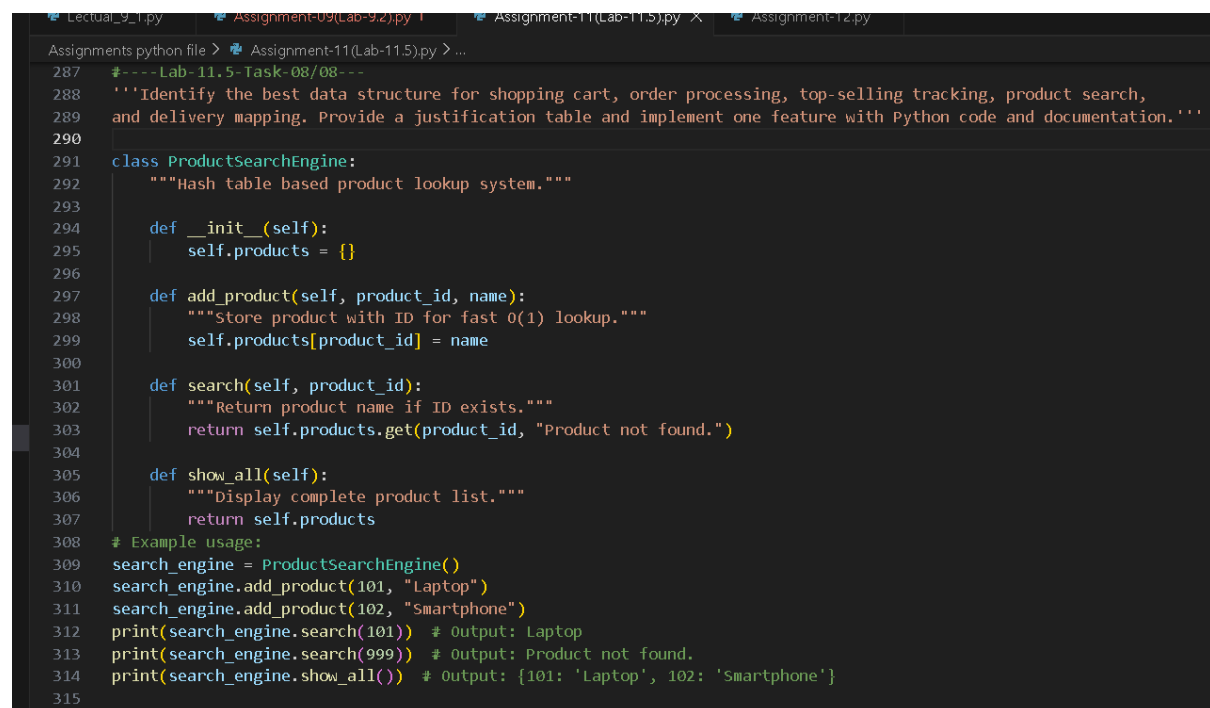
This priority queue ensures that emergency vehicles pass first.
heapq naturally sorts items by priority where lower numbers represent higher urgency.
Ambulances and police vehicles get processed before normal cars.
The structure solves real-life emergency routing realistically.
The system handles multiple priority levels efficiently.
This provides a fast and fair traffic management solution.

Tack-08

Final Optimal Prompt

Identify the best data structure for shopping cart, order processing, top-selling tracking, product search, and delivery mapping. Provide a justification table and implement one feature with Python code and documentation.

Code Screenshot



```
287 #---Lab-11.5-Task-08/08---
288 '''Identify the best data structure for shopping cart, order processing, top-selling tracking, product search,
289 and delivery mapping. Provide a justification table and implement one feature with Python code and documentation.'''
290
291 class ProductSearchEngine:
292     """Hash table based product lookup system."""
293
294     def __init__(self):
295         self.products = {}
296
297     def add_product(self, product_id, name):
298         """Store product with ID for fast O(1) lookup."""
299         self.products[product_id] = name
300
301     def search(self, product_id):
302         """Return product name if ID exists."""
303         return self.products.get(product_id, "Product not found.")
304
305     def show_all(self):
306         """Display complete product list."""
307         return self.products
308
309 # Example usage:
310 search_engine = ProductSearchEngine()
311 search_engine.add_product(101, "Laptop")
312 search_engine.add_product(102, "Smartphone")
313 print(search_engine.search(101)) # Output: Laptop
314 print(search_engine.search(999)) # Output: Product not found.
315 print(search_engine.show_all()) # Output: {101: 'Laptop', 102: 'Smartphone'}
```

Output Screenshot

```
● PS D:\6th Sem 2025-26\AI-Assisted Coding> & "D:\p  
ile/Assignment-11(Lab-11.5).py"  
Laptop  
Product not found.  
{101: 'Laptop', 102: 'Smartphone'}  
○ PS D:\6th Sem 2025-26\AI-Assisted Coding> █  
●
```

Explanation/Justification/Observation (100 words / 5 – 6 sentence)

Hash tables allow instant access to products by unique ID.

add_product() stores items using O(1) dictionary operations.

search() retrieves product names without scanning the entire list.

This design scales efficiently for millions of products.

The structure is perfect for e-commerce where fast lookup improves user experience.

This implementation is clean, simple, and highly efficient.